

Agentic clients: plan versus execute, MCP, and the Chrome extension

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 2 of 12 · English notes

The deepest architecture point of 5 August: **Cowork and Claude Code are agentic clients**. You give a task that cannot finish in one LLM shot. The model returns a *plan*. The client walks the plan, mixing tool/HTTP steps (often no extra model call) with further LLM steps (extract, generate). Chat can use tools; Vijay still reserved “agentic solution” for the products that run that loop against local and connector tools.

Worked example: weather plus image background

Prompt: upload a photo; get Bengaluru weather; apply it as the background. One Messages call cannot honestly do all of that. Typical plan the model might emit:

1. Fetch current weather via a tool (default web search, a weather connector, or your own API).
2. Reduce a verbose JSON blob to a keyword (rainy / sunny) — usually another LLM pass.
3. Generate or composite the background — another generative call.

Who decided a tool was required? **The LLM, in the planning turn**. Who performs the HTTP? **Cowork’s HTTP client**, using URL/body already specified in the plan. That is why Vijay said “no LLM in this task” for step 1 *after* the plan exists. Students who heard “no LLM at all” were stuck; Anand’s question unstuck the room: the planner is always a model. Client-side tool calls are cheaper than models that invoke tools inside the provider. SDK weeks will contrast those two implementations.

Connectors, MCP, and custom tools

Vyomkesh: are connectors the same as MCP servers? Vijay: **same approach**. Cowork does not expose a developer “enable MCP” toggle; the catalogue *is* MCP. Custom MCP can be added via settings / an MCP file—he parked the how-to for SDK week because Cowork is aimed at non-developers clicking UI. If you have no public connector for a semiconductor parts database, wrap it in a Lambda (or any API) and attach that function as a tool. Scraping: enable a crawler connector (Apify was on screen), pass a URL, get extracted text. Switching Figma → Adobe XD is the textbook reason to prefer MCP over a hard-coded vendor SDK inside your agent.

Pattern	Realtime?	Best when	Drawback Vijay stressed
Tool / connector / MCP	Yes, if the backend is live	Third-party systems, vendor swaps, weather	Must be allow-listed; OAuth consent
RAG	No — pre-indexed corpus	Stable internal docs after vuln review	Stale the moment the wiki moves
Retrain / SLM fine-tune	No	Tiny domain models, not frontier LLMs	Cost and time
Web search	Yes	Public internet after cutoff	Tokens and messier provenance

Chrome extension as another agentic client

For browser-native chores (LinkedIn scan, “manual” test of a web app, blogger workflows), Vijay installed the **Claude Chrome extension** from the Web Store (publisher claude.com). Authorise it; you get a Chat-like pane bound to the browser. If you are already logged into a site, the extension can drive that session—he called that **dangerous** and org-sensitive. In class it was **beta**: most enterprises will block it. It is not Copilot: Copilot (in his contrast) talks to an LLM; this extension can *execute* browser steps. He also warned not to mix Office-in-Chrome automation with a separate web flow as if they were one agent without checking product limits.

UST INSTALL POLICY

Whether R&D VDI may load the extension is an IT question, not a trainer promise. Personal Pro can demo it. Do not put client credentials into a beta browser agent.

You cannot download Sonnet

A closing Q&A tried to treat Claude like early GPT weights you copied to disk. Vijay: you **access** models; you do not take them home. That keeps the client–server story honest for the exam.

Exam phrasing

- Plan = model; execute = client (Cowork, Code, or Chrome agent).
- HTTP tool step after a plan ≠ “the model is stateless so nothing happened locally.”
- Connectors ≈ MCP in Claude products; custom APIs are still tools.
- Beta browser agents are a capability question, not a default enterprise control.

Bridge into Claude Code

The same plan/execute loop is how Claude Code will run tests, read logs (terminal as a tool), and edit many files. Module 03 installs that client. If you only remember one sentence from 5 August: **agentic means many model-and-tool turns orchestrated by a client that has hands on your world.**

Official documentation

• Tool use overview

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Planning versus acting with tools in the API.

• MCP

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

The protocol behind Cowork connectors.

• Managed Agents quickstart

<https://platform.claude.com/docs/en/managed-agents/quickstart>

Platform-side agent narrative as products evolve.

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

The agentic coding client this loop is heading toward.

- **Claude desktop support**

<https://support.claude.com/>

Chrome / desktop extras as they leave beta.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.